

Implementación de las compuertas NOR y XNOR mediante neuronas Hebb y Perceptrón

Introducción

En este ejercicio se implementaron las operaciones lógicas NOR y XNOR mediante modelos neuronales sencillos. La compuerta NOR se entrenó con una neurona basada en la regla de Hebb y con un perceptrón. La función XNOR se construyó como una red formada por cuatro neuronas NOR, debido a que el modelo utilizado en el notebook no resuelve esta operación con una sola neurona.

El desarrollo se realizó en Python con PyTorch. Las entradas y salidas se representaron de manera bipolar, por lo que el valor lógico 0 se sustituyó por -1 y el valor lógico 1 se conservó como 1.

1. Fundamento lógico de las compuertas

1.1 Compuerta NOR

NOR es la negación de la operación OR. Su salida vale 1 únicamente cuando ambas entradas son 0. En cualquier otro caso la salida es 0.

$$\text{NOR}(A, B) = \neg(A \vee B)$$

A	B	A OR B	A NOR B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

1.2 Compuerta XNOR

XNOR, también llamada equivalencia lógica, produce una salida igual a 1 cuando las dos entradas tienen el mismo valor. Por lo tanto, los casos 00 y 11 son positivos, mientras que 01 y 10 producen 0.

$$\text{XNOR}(A, B) = (A \wedge B) \vee (\neg A \wedge \neg B)$$

A	B	A XNOR B
0	0	1
0	1	0
1	0	0
1	1	1

2. Representación de las entradas

El código no trabaja directamente con 0 y 1, sino con valores bipolares. Esta codificación permite utilizar una función de activación con resultados -1 y 1.

Valor binario	Valor bipolar
0	-1
1	1

Los conjuntos utilizados en el notebook fueron:

```
S = torch.tensor([
    [ 1, 1],
    [ 1, -1],
    [-1, 1],
    [-1, -1]
])

T_NOR = torch.tensor([-1, -1, -1, 1])
T_XNOR = torch.tensor([ 1, -1, -1, 1])
```

3. Modelo de neurona y fórmulas utilizadas

3.1 Entrada neta

Cada neurona combina las entradas con sus pesos y agrega un término independiente llamado bias. La entrada neta determina posteriormente la salida de la función de activación.

$$y_{in} = x_1w_1 + x_2w_2 + b$$

3.2 Regla de Hebb

La regla de Hebb actualiza los parámetros usando cada patrón de entrada y su salida objetivo. En la implementación se parte de pesos y bias iguales a cero.

$$w(\text{nuevo}) = w(\text{anterior}) + x \cdot t$$

$$b(\text{nuevo}) = b(\text{anterior}) + t$$

Fragmento principal del entrenamiento Hebb:

```
for i in range(len(S)):
    x = S[i].to(dtype=torch.float32)
    t = T[i].to(dtype=torch.float32)

    self.w = self.w + x * t
    self.b = self.b + t
```

3.3 Regla del perceptrón

El perceptrón calcula una predicción y modifica sus pesos únicamente cuando la salida obtenida es distinta del objetivo. En el ejercicio se utilizaron una tasa de aprendizaje igual a 1 y un umbral igual a 1.

$$w(\text{nuevo}) = w(\text{anterior}) + \alpha \cdot x \cdot t$$

$$b(\text{nuevo}) = b(\text{anterior}) + \alpha \cdot t$$

Función de activación usada por el perceptrón:

Condición	Salida
$y_{in} > \theta$	1
$y_{in} < -\theta$	-1
$-\theta \leq y_{in} \leq \theta$	0

4. Representación del funcionamiento neuronal

4.1 Neurona utilizada para NOR

Después del entrenamiento, tanto Hebb como el perceptrón obtuvieron pesos negativos. Esto es coherente con la operación NOR: cuando alguna entrada toma el valor positivo 1, su contribución reduce la activación de la neurona. La salida positiva se conserva para el caso en el que ambas entradas representan 0.

$$\text{Entrada neta: } y_{in} = A \cdot w_1 + B \cdot w_2 + b$$

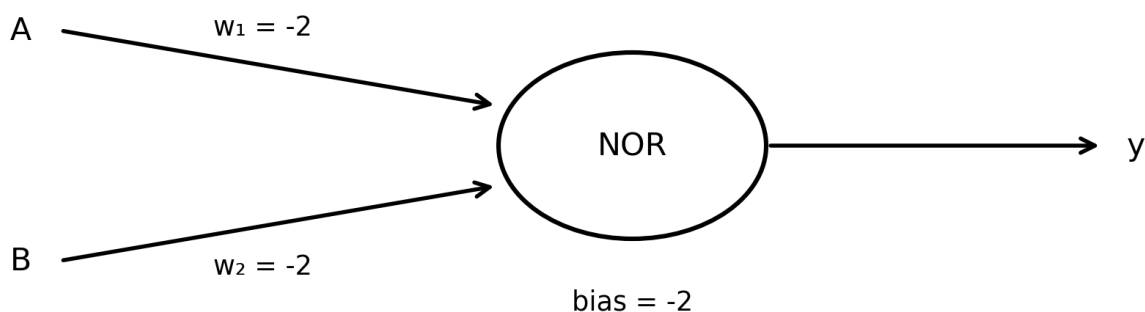


Figura 1. Modelo de la neurona NOR con los parámetros obtenidos en el entrenamiento.

4.2 Red neuronal utilizada para XNOR

La función XNOR fue implementada con cuatro copias de una neurona NOR ya entrenada. La primera neurona calcula una salida intermedia n_1 . Dos neuronas de la segunda capa combinan n_1 con las entradas originales, y una última neurona NOR genera la salida XNOR.

Arquitectura multicapa utilizada para implementar XNOR

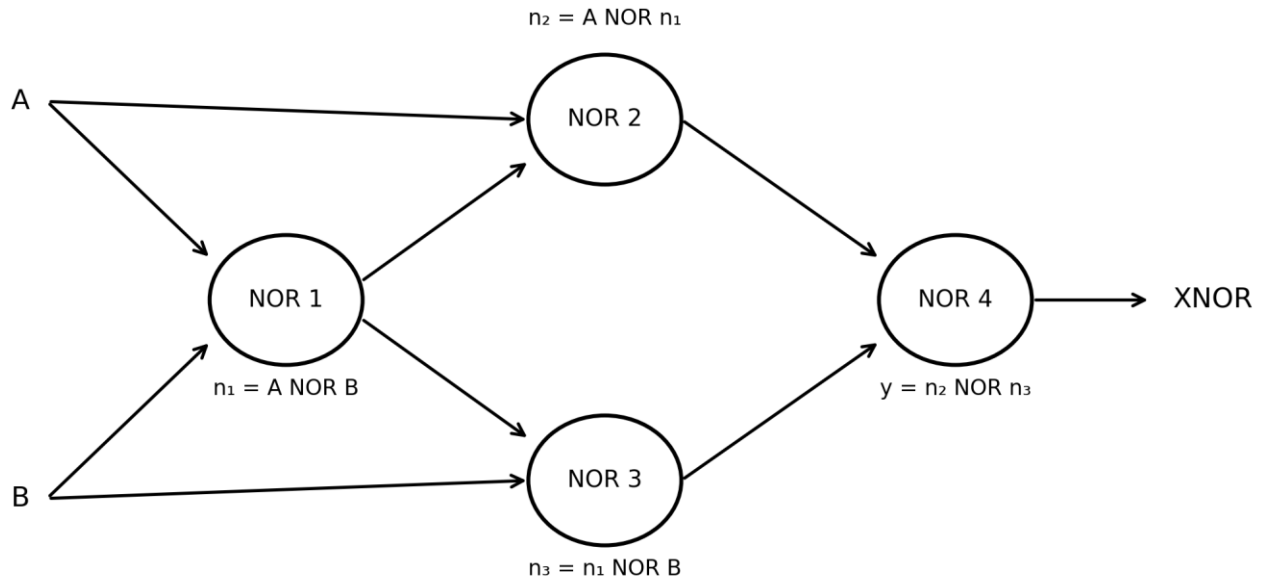


Figura 2. Red multicapa de cuatro neuronas NOR utilizada para construir XNOR.

5. Implementación principal

5.1 Entrenamiento de NOR

Se crearon dos modelos independientes: uno con la clase `HebbNeuron` y otro con la clase `PerceptronNeuron`.

```
hebb_nor = HebbNeuron(input_size=2)
hebb_nor.train(S=S, T=T_NOR)

perceptron_nor = PerceptronNeuron(input_size=2)
convergio = perceptron_nor.train(
    S=S,
    T=T_NOR,
    threshold=1,
    learning_rate=1,
    max_epochs=100
)
```

5.2 Construcción de la red XNOR

La clase `XNORNetwork` copia cuatro veces la neurona NOR entrenada y ejecuta las operaciones en orden.

```
n1 = self.nor_1.forward(torch.tensor([a, b]))
n2 = self.nor_2.forward(torch.tensor([a, n1]))
n3 = self.nor_3.forward(torch.tensor([n1, b]))
y = self.nor_4.forward(torch.tensor([n2, n3]))
```

6. Resultados obtenidos

6.1 Resultados de NOR con Hebb

A	B	Esperado	Obtenido
1	1	0	0
1	0	0	0
0	1	0	0
0	0	1	1

Parámetros finales: $w_1 = -2$, $w_2 = -2$ y bias = -2. Exactitud: 4 de 4 casos, equivalente al 100 %.

6.2 Resultados de NOR con perceptrón

Parámetro	Resultado
Convergencia	Sí
Épocas	2
w_1	-2
w_2	-2
Bias	-2

El perceptrón reprodujo correctamente la tabla de verdad de NOR y alcanzó una exactitud del 100 %. El umbral empleado durante el entrenamiento fue 1.

6.3 Resultados internos de la red XNOR

A	B	n_1	n_2	n_3	Esperado	Hebb	Perceptrón
1	1	0	0	0	1	1	1
1	0	0	0	1	0	0	0
0	1	0	1	0	0	0	0
0	0	1	0	0	1	1	1

Las dos versiones de la red XNOR, una construida con neuronas Hebb y otra con perceptrones NOR, obtuvieron los cuatro casos correctos. La exactitud final fue del 100 % para ambas.

7. Análisis de los resultados

- La compuerta NOR pudo aprenderse con una sola neurona, porque los patrones positivos y negativos fueron clasificados correctamente por una combinación lineal de pesos y bias.
- Los pesos negativos de NOR indican que las entradas positivas actúan como señales inhibitorias para la salida.
- La regla de Hebb obtuvo directamente $w_1 = -2$, $w_2 = -2$ y bias = -2 después de procesar los cuatro patrones.
- El perceptrón llegó a los mismos parámetros y finalizó el entrenamiento después de dos épocas.
- XNOR no se ejecutó como una sola neurona en el notebook; se representó mediante una red de cuatro neuronas NOR conectadas en varias capas.
- Las tablas obtenidas coincidieron con las salidas esperadas en todos los casos evaluados.

Conclusión

El ejercicio permitió comprobar que las compuertas lógicas pueden representarse mediante modelos neuronales simples, siempre que se considere la estructura adecuada. NOR se resolvió con una sola neurona utilizando tanto la regla de Hebb como el perceptrón, y ambos métodos alcanzaron una exactitud del 100 %. Los parámetros finales fueron $w_1 = -2$, $w_2 = -2$ y bias = -2; en el perceptrón se empleó un umbral de 1 y el aprendizaje terminó en dos épocas.

Para XNOR fue necesario combinar cuatro neuronas NOR. Esta arquitectura permitió generar salidas intermedias y construir la equivalencia lógica de manera correcta. El resultado final también fue del 100 %, lo que demuestra que una función más compleja puede obtenerse al conectar varias unidades neuronales sencillas en una red.